



Advanced Traffic (20–551) Class Project

Project Description: Network Traffic Simulation

Goal: Students will design, implement, and analyze a traffic network simulation using Python. The simulation represents vehicle movement in a network of nodes, edges, blocks, and intersections with traffic lights. After completing the Python-based simulation, the same network will be modeled and simulated inside SUMO to compare results.

Libraries Allowed

Students may use any of the following (or similar) Python packages:

- **pandas** for data reading and handling
- **matplotlib** for plots
- **networkx** for graph/network structure
- **simpy** for event simulation
- **sympy** for formulas or any other library

Part 1: Network Structure

The input consists of:

- **Node file** – list of nodes with IDs and optional coordinates.
- **Edge file** – for each edge: start node, end node, length, number of lanes (optional).
- **Demand file** – each row indicates:
 - vehicle ID,
 - start time,
 - origin node,
 - destination node.

Network configuration:

- Each edge is divided into blocks of 100 meters.
- Each block has capacity of about 20 vehicles.
- Vehicles may move one block every 5 simulation steps.
- There may be more than one lane per edge (optional part).

Part 2: Vehicle Simulation Rules

- Simulation runs in discrete time steps (1 second per step).
- Vehicles are not required to enter only at times 5, 10, 15...; they may appear at any time (e.g., $t = 7$).
- If a block is full, vehicles must wait and accumulate delay time.
- Each intersection has a traffic signal:
 - fixed-time configuration,
 - or adaptive, (optional)
 - or any custom logic.
- Vehicles must obey signal states. Red signals cause queue formation.
- Vehicles may choose:
 - a fixed path at start,
 - or a dynamic path that changes if streets become congested (optional).
- Automated/connected vehicles may be included to improve efficiency (optional).

Part 3: Outputs (Python)

During the simulation, the program must produce:

- Total wait time of all vehicles:
 - at intersections (red light delays),
 - inside edges due to crowded blocks.
- Trajectory plots for selected lanes.
- Queue length plots for intersections over simulation time.
- Summary statistics (mean delays, travel times, throughput).

Part 4: SUMO Implementation

After completing the Python simulation:

- The same network (nodes and edges) must be exported or recreated in SUMO.
- The same demand file must be used for vehicle input.
- Traffic lights should follow the same logic as Python implementation.
- SUMO output will be collected in real-time (more realistic dynamics).
- Final comparison between:
 - Python simulation,
 - SUMO simulation.

Final Deliverables

1. Python implementation with clear OOP structure (vehicles, network components, traffic lights, statistics).
2. SUMO network configuration (`.net.xml`, `rou.xml`, traffic-light definitions).
3. Plots: delays, queues, trajectories.
4. A final mini-report including:
 - methodology,
 - code explanation,
 - results from Python,
 - results from SUMO,
 - comparison and conclusion.

Grading

- Correctness of simulation logic,
- Object-oriented design quality,
- Ability to handle large demand without errors,
- Quality of plots and interpretation,
- SUMO results and comparison.